

Fonctions Avancées, Paramétrage et Gestion Robuste des Erreurs

1. Cadrage Opérationnel

L'objectif de cet atelier est de transformer des commandes impératives en outils d'administration fiables, paramétrables et conformes aux exigences d'un environnement de production.

Vous apprendrez à :

1. Construire une fonction modulaire de journalisation respectant la nomenclature officielle et le principe DRY.
2. Déclarer un bloc param() complet avec typage fort, valeurs par défaut et commutateurs ([switch]).
3. Exploiter les apports de l'attribut [CmdletBinding()] pour activer les paramètres communs et le contrôle strict des arguments.
4. Convertir des erreurs non-terminantes en exceptions bloquantes via -ErrorAction Stop pour rendre effectifs les blocs try / catch.
5. Extraire les informations utiles d'un incident via \$_.Exception.Message et garantir la libération de ressources avec finally.
6. Concevoir un script industriel d'audit de répertoire sécurisé nommé Audit-RepertoireSecurise.ps1.

2. Exercice 1 : Conception d'une Fonction de Journalisation (Write-LogMessage)

Objectif

Créer une fonction interne réutilisable permettant d'écrire des messages d'audit horodatés à l'écran et/ou dans un fichier, sans dupliquer de code.

Instructions

1. **Création de la fonction et nomenclature :**
 - Déclarez une fonction nommée Write-LogMessage (respect du modèle Verbe-Nom approuvé).
 - Ajoutez l'attribut [CmdletBinding()] au-dessus de son bloc de paramètres.
2. **Déclaration des paramètres :**
 - \$Message : de type [string], obligatoire ([Parameter(Mandatory = \$true)]).
 - \$Niveau : de type [string], obligatoire, restreint strictement aux valeurs "INFO", "WARNING" et "ERROR" via l'attribut [ValidateSet()].
 - \$CheminFichier : de type [string], facultatif, avec pour valeur de repli \$null.

3. Traitement et aiguillage (switch) :

- Récupérez la date et l'heure actuelles au format yyyy-MM-dd HH:mm:ss.
- Construisez la chaîne formatée suivante :
[Date-Heure] [NIVEAU] Message
- À l'aide d'une instruction switch (\$Niveau), colorez le message dans la console :
 - Si "INFO" : texte affiché en vert (-ForegroundColor Green).
 - Si "WARNING" : texte affiché en jaune (-ForegroundColor Yellow).
 - Si "ERROR" : texte affiché en rouge (-ForegroundColor Red).

4. Persistance conditionnelle :

- Si le paramètre \$CheminFichier est renseigné (non nul et non vide), inscrivez la ligne formatée dans le fichier cible avec un encodage UTF-8 à l'aide de Add-Content.

5. Validation :

- Testez l'appel nominal :
Write-LogMessage -Message "Service démarré avec succès." -Niveau "INFO"
- Vérifiez que la saisie d'un niveau invalide est immédiatement rejetée avant exécution :
Write-LogMessage -Message "Test" -Niveau "DEBUG"
- Testez l'écriture simultanée en console et dans un fichier temporaire :
Write-LogMessage -Message "Espace disque bas." -Niveau "WARNING"
-CheminFichier "\$HOME\audit.log"

3. Exercice 2 : Industrialisation et Rôle de [CmdletBinding()]

Objectif

Mesurer concrètement la différence de comportement entre un script non contrôlé et un script exploitant l'attribut [CmdletBinding()].

Instructions

1. Expérimentation du comportement permissif (sans [CmdletBinding()]) :

- Créez un fichier temporaire test_sans_binding.ps1 contenant :

```
param(  
    [string]$Repertoire = "C:\Temp"  
)  
Write-Host "Cible : $Repertoire"
```
- Exécutez le script en simulant une faute de frappe sur le nom du paramètre :
.test_sans_binding.ps1 -Repertoire "C:\Windows"
- **Question d'analyse** : Quelle valeur a pris \$Repertoire ? Le script a-t-il signalé l'erreur

ou l'a-t-il masquée ?

2. Sécurisation avec [CmdletBinding()] :

- Créez un fichier test_avec_binding.ps1 et ajoutez [CmdletBinding()] immédiatement au-dessus de param().
- Relancez la même commande avec la faute de frappe.
- **Question d'analyse :** Quel message d'erreur PowerShell affiche-t-il désormais ? Pourquoi ce comportement est-il indispensable pour un script de production ?

3. Exploitation des paramètres communs hérités :

- Dans test_avec_binding.ps1, ajoutez une instruction de diagnostic :
Write-Verbose "Initialisation de l'analyse sur le dossier : \$Repertoire"
- Lancez le script sans paramètre particulier, puis lancez-le avec le commutateur -Verbose.
- Constatez l'affichage conditionnel du message sans avoir eu à coder de logique spécifique.

4. Exercice 3 : Typologie des Erreurs et Protection Try / Catch / Finally

Objectif

Comprendre pourquoi try / catch ne réagit pas par défaut sur la plupart des cmdlets PowerShell, et implémenter la conversion via -ErrorAction Stop.

Instructions

1. L'échec de la capture par défaut (Erreur non-terminante) :

- Exécutez le bloc suivant dans votre console :

```
try {  
    Get-ChildItem -Path "C:\DossierTotalementInexistant"  
    Write-Host "Ligne après la commande en échec."  
}  
catch {  
    Write-Host "Le bloc catch a intercepté l'erreur !" -ForegroundColor Yellow  
}
```
- **Constat :** Le bloc catch s'est-il déclenché ? Pourquoi PowerShell a-t-il continué l'exécution malgré le texte rouge ?

2. Forçage de l'interception (-ErrorAction Stop) :

- Modifiez le bloc précédent en ajoutant -ErrorAction Stop sur Get-ChildItem.
- Constatez le routage direct vers le bloc catch et l'interruption des instructions suivantes du try.

3. Extraction précise de l'exception :

- Dans le bloc catch, affichez uniquement le texte de l'incident sans la trace technique

```
d'appel :  
catch {  
    Write-Host "Incident intercepté : $($_.Exception.Message)" -ForegroundColor Red  
}
```

4. Garantie d'exécution avec finally :

- Ajoutez un bloc finally à la structure :

```
finally {  
    Write-Host "Fin de la tentative d'accès au stockage." -ForegroundColor Gray  
}
```
- Testez l'exécution sur un chemin inexistant, puis sur un chemin valide (C:\Windows).
- Vérifiez que les instructions du finally s'exécutent dans les deux scénarios.

5. Exercice 4 : Cas de Synthèse (Audit-RepertoireSecurise.ps1)

Objectif

Rassembler l'ensemble des notions au sein d'un outil d'infrastructure opérationnel.

Énoncé du besoin

L'équipe d'exploitation souhaite disposer d'un script permettant de recenser tous les fichiers d'un dossier dont la taille dépasse un certain seuil en mégaoctets (Mo), tout en assurant une traçabilité complète des erreurs (chemins introuvables, autorisations insuffisantes).

Spécifications fonctionnelles et techniques

1. En-tête et Métadonnées :

- Le script doit débuter par l'attribut [CmdletBinding()].
- Le bloc param() doit exposer :
 - [string]\$DossierSource = "C:\Windows" : chemin du répertoire à analyser.
 - [int]\$SeuilTailleMo = 10 : taille minimale des fichiers ciblés (en Mo).
 - [string]\$FichierJournal = "\$HOME\audit_fichiers.log" : chemin du journal d'audit.
 - [switch]\$InclureSousDossiers : active l'analyse récursive si présent.

2. Fonction de journalisation intégrée :

- Intégrez la fonction Write-LogMessage (développée à l'Exercice 1) en tête de script.
- Toutes les informations, avertissements et erreurs devront impérativement transiter par cette fonction.

3. Logique de contrôle et résilience (try / catch / finally) :

- Tracez le démarrage de l'analyse avec le niveau "INFO".
- Dans un bloc try :
 - Vérifiez l'existence du dossier source avec Test-Path. S'il est absent, levez une

- exception explicite via throw "Le répertoire spécifié n'existe pas.".
- Récupérez les fichiers à l'aide de Get-ChildItem -File en utilisant -ErrorAction Stop. Prenez en compte le commutateur \$InclureSousDossiers (paramètre -Recurse).
 - Filtrez les fichiers dont la taille (Length) est supérieure à \$SeuilTailleMo * 1MB.
 - Pour chaque fichier identifié :
 - Calculez sa taille en mégaoctets arrondie à deux décimales via $[math]::Round($_.Length / 1MB, 2)$.
 - Consignez un avertissement ("WARNING") précisant le nom du fichier et son poids.
 - Émettez un objet personnalisé [PSCustomObject] dans le pipeline contenant les propriétés : Nom, CheminComplet, TailleMo, DerniereModification.
 - Dans le bloc catch :
 - Interceptez l'anomalie et journalisez un message de niveau "ERROR" reprenant \$_.Exception.Message.
 - Dans le bloc finally :
 - Journalisez un message de niveau "INFO" marquant l'achèvement de la procédure d'audit.

6. Protocole de Validation et Recette

Validez la conformité de votre script Audit-RepertoireSecurise.ps1 à l'aide des quatre cas d'usage suivants :

Test 1 : Exécution nominale avec paramètres par défaut

.\Audit-RepertoireSecurise.ps1

- **Résultat attendu :** Analyse du dossier C:\Windows, détection des fichiers volumineux (> 10 Mo) à la racine, consignation des alertes dans la console et dans le fichier journal, restitution des objets structurés.

Test 2 : Saisie d'arguments personnalisés et récursion

.\Audit-RepertoireSecurise.ps1 -DossierSource "C:\Windows\System32" -SeuilTailleMo 50 -InclureSousDossiers

- **Résultat attendu :** Parcours récursif, filtrage strict sur les fichiers de plus de 50 Mo, traçabilité horodatée.

Test 3 : Gestion d'un répertoire inexistant (Test de résilience)

.\Audit-RepertoireSecurise.ps1 -DossierSource "Z:\PartageInexistant"

- **Résultat attendu :** Aucune erreur rouge brute en console ; interception par le bloc catch, écriture d'une ligne d'erreur formatée dans le log via Write-LogMessage, et exécution du bloc finally.

Test 4 : Contrôle de l'héritage des paramètres communs

.\Audit-RepertoireSecurise.ps1 -DossierSource "C:\Windows" -Verbose

- **Résultat attendu :** Présence des traces détaillées issues de Write-Verbose en complément des messages de log standards.

7. Grille d'Évaluation du TP

Critère d'évaluation	Indicateur de conformité	Validé
Nomenclature et DRY	Fonction nommée en Verbe-Nom approuvé, réutilisée pour tous les messages du script.	[]
Typage et Validation	Paramètres typés ([string], [int], [switch]), utilisation conforme de [ValidateSet()].	[]
Contrôle avancé (CmdletBinding)	Présence de [CmdletBinding()], rejet automatique des arguments mal orthographiés.	[]
Conversion d'erreurs	Utilisation systématique de -ErrorAction Stop sur les opérations critiques.	[]
Gestion d'exception	Bloc try / catch fonctionnel, extraction de \$_.Exception.Message sans trace parasite.	[]
Garantie d'exécution	Bloc finally présent et validé	[]

	aussi bien en cas de succès que d'incident.	
Aiguillage lisible	Utilisation de l'instruction switch pour traiter les niveaux de criticité.	[]
Intégrité de sortie	Émission d'objets structurés [PSCustomObject] réutilisables dans le pipeline.	[]